

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Введение в нереляционные базы данных»
Тема: Сервис выбора земельных участков для купли-продажи

Студенты гр. 3343

Преподаватель

Атоян М. А.
Гребнев Е. Д.
Малиновский А. А.
Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Студенты

Атоян М. А.

Гребнев Е. Д.

Малиновский А. А.

Группа 3343

Тема работы: Сервис выбора земельных участков для купли-продажи

Исходные данные: Задача - создать сервис, в котором можно просматривать и визуализировать объявления о купле-продаже участков. Для участков необходимо составлять аналитику с точки зрения их привлекательности или наоборот - негативных факторов для покупки.

Предлагаемая СУБД : MongoDB

Содержание пояснительной записки:

- «Введение»
- «Качественные требования к решению»
- «Сценарии использования»
- «Модель данных»
- «Разработанное приложение»
- «Вывод»
- «Приложения»
- «Используемая литература»

Предполагаемый объем пояснительной записки:

Не менее 10 страниц.

Дата выдачи задания: 10.02.2026

Дата сдачи реферата: 15.05.2026

Дата защиты реферата: 15.05.2026

Студенты	_____	Атоян М. А. Гребнев Е. Д. Малиновский А. А.
Преподаватель	_____	Заславский М.М.

АННОТАЦИЯ

В рамках данного курса предполагалось разработать приложение на одну из заданных тем, использующее нереляционную базу данных, в команде из трёх человек. Была выбрана тема создания приложения для работы с базой данных объявлений о продаже земельных участков с автоматическим расчётом привлекательности участка по геолокации и текстовому описанию. Для хранения карточек участков, инфраструктурных объектов и пользователей использовалась MongoDB. Исходный код хранится в репозитории проекта: <https://github.com/moevm/nsql1h26-land>.

ABSTRACT

This course involved developing an application on one of the given topics using a non-relational database in a team of three. The chosen topic was an application for working with a database of land plot listings featuring an automatic plot attractiveness score based on geolocation and text description. MongoDB was used to store plots, infrastructure objects and users. The source code is stored in the project repository: <https://github.com/moevm/nsql1h26-land>.

Оглавление

1. Введение.....	6
2. Качественные требования к решению.....	8
3. Сценарии использования.....	9
4. Модель данных.....	16
5. Разработанное приложение.....	28
6. Выводы.....	31
7. Приложения.....	33
8. Используемая литература.....	35

1. Введение

Актуальность решаемой проблемы

Рынок частных земельных участков в Ленинградской области и Санкт-Петербурге остаётся одним из самых активных сегментов недвижимости: на классифайдах вроде Авито одновременно находится несколько тысяч объявлений. При этом качество объявлений сильно неоднородно — описание может быть как развёрнутым, так и почти пустым, цена и реальные характеристики (доступ к коммуникациям, удалённость от метро, наличие негативных объектов вроде свалок или промзон) часто не сопоставимы напрямую.

Покупатель тратит много времени на фильтрацию: он вынужден вручную сопоставлять координаты участка с картой инфраструктуры, читать длинные описания, чтобы понять, есть ли газ или дом, и оценивать, не находится ли участок рядом с шумной магистралью. Существующие классифайды не предоставляют интегральной оценки привлекательности участка, не учитывают близость к негативным факторам и не позволяют семантически искать по запросам типа «тихий участок рядом с метро с домом».

Разработанное в рамках курса приложение отвечает этим требованиям: оно автоматически рассчитывает скоринг участка по геолокации (близость метро, школ, больниц, магазинов, ПВЗ, остановок и негативных объектов) и по семантике описания (15 признаков, извлекаемых из текста через embeddings).

Постановка задачи

Необходимо реализовать веб-приложение для работы с базой данных объявлений о продаже земельных участков с использованием MongoDB как основной СУБД. Приложение должно позволять:

- хранить и обновлять карточки участков, инфраструктурных объектов и пользователей;
- выполнять поиск, фильтрацию и сортировку каталога;
- отображать участки на интерактивной карте;

- автоматически рассчитывать оценку привлекательности по гео- и текстовым признакам;
- поддерживать массовый импорт и экспорт данных в формате JSON;
- разграничивать роли «пользователь» и «администратор».

Предлагаемое решение

Реализуется веб-приложение с трёхзвенной контейнеризованной архитектурой: React-фронтенд, FastAPI-бэкенд с гексагональной архитектурой (слои domain, infrastructure, interfaces) и MongoDB. Геопоиск выполняется через встроенный \$geoNear по 2dsphere-индексу. Семантический поиск реализован двухстадийно: BM25-ретривал на стороне приложения + Jina Reranker для финального ранжирования топ-100.

2. Качественные требования к решению

Для разработанного решения сформулирован следующий перечень качественных требований:

1. Функциональные требования.

- Хранение карточек участков, инфраструктурных объектов и пользователей в единой нереляционной СУБД.
- Поиск, фильтрация и сортировка каталога по числовым диапазонам (цена, площадь, цена за сотку, сроки), по локации, а также произвольный текстовый поиск с семантическим ранжированием.
- Отображение участков на интерактивной геокарте с кластеризацией и цветовой индикацией привлекательности.
- Автоматический расчёт интегрального счёта участка по геопризнакам (расстояния до 8 типов инфраструктуры и негативных объектов) и по 15 текстовым признакам.
- Поддержка ручного CRUD объявлений и пакетного импорта / экспорта данных в формате JSON.
- Разграничение ролей «пользователь» и «администратор» с раздельным набором операций.

2. Нефункциональные требования.

- Развёртывание системы одной командой через docker-compose без ручной настройки внешних сервисов.
- Время отклика на каталог и страницу деталей — не более 1 секунды на типовой выборке (≈ 2000 объявлений).
- Гибкость схемы: добавление нового типа инфраструктуры или текстового признака не должно требовать миграций.
- Безопасное хранение паролей (PBKDF2-SHA256) и токен-ориентированная аутентификация (JWT).
- Возможность работы без авторизации в режиме «только чтение» (каталог, карта, детали).

3. Сценарии использования

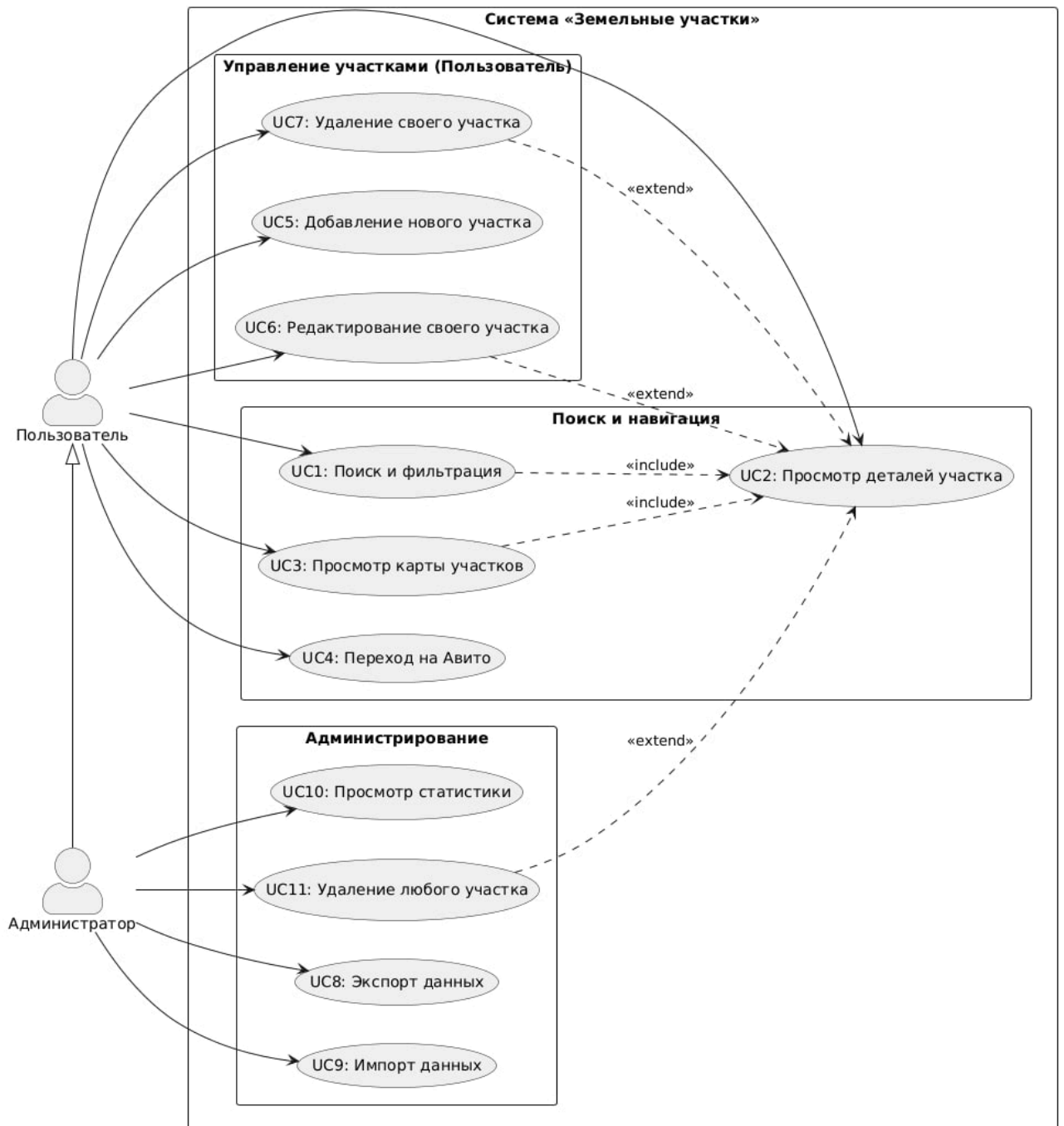


Рисунок 1. Сценарии использования

Сценарии использования для задачи

В системе выделены две роли: Пользователь и Администратор (наследует все возможности пользователя). Ниже приведены ключевые юзкейсы, сгруппированные по требуемым категориям ИДЗ.

I. Импорт данных

UC9 — массовый импорт данных (администратор)

Предусловия: администратор авторизован; открыта страница «Панель данных»; подготовлен корректный JSON-файл.

Шаг	Пользователь	Система
1	Нажимает «Импорт JSON» и выбирает .json-файл	Открывает диалог выбора файла
2	Подтверждает выбор файла	Распознаёт формат: массив объектов либо объект с ключом plots / data
3	Ожидает завершения операции	Загружает данные, для участков рассчитывает фичи, расстояния и скоры; для инфраструктуры заменяет коллекцию и пересчитывает score в фоне
4	Видит результат	Показывает сообщение об успешном импорте и обновляет статистику

Альтернативный сценарий: некорректный формат файла → система показывает ошибку парсинга; ошибка сервера → отображается сообщение об ошибке.

UC5 — ручное добавление участка (пользователь)

Предусловия: пользователь авторизован.

Шаг	Пользователь	Система
1	Открывает форму «Новый участок»	Отображает форму с интерактивной картой Leaflet
2	Нажимает на точку на карте для указания координат	Сохраняет координаты, ставит маркер

Шаг	Пользователь	Система
3	Заполняет название, описание, цену, площадь, локацию, адрес, URL	Валидирует поля по схеме на лету
4	Нажимает «Добавить участок»	Автоматически рассчитывает 15 текстовых признаков, расстояния до 8 типов инфраструктурных объектов, скоры и первую точку истории цены
5	Получает страницу созданного участка	Перенаправляет на детали участка

Альтернативный сценарий: координаты не указаны → ошибка «Укажите точку на карте»; название не заполнено → форма не отправляется; ошибка сервера → отображается сообщение об ошибке.

II. Представление данных

UC1 — поиск и фильтрация участков (пользователь)

Предусловия: сервис доступен; в базе есть объявления.

Шаг	Пользователь	Система
1	Открывает страницу «Каталог»	Отображает каталог с поисковой строкой, фильтрами, сортировкой, пагинацией и карточками
2	Вводит произвольный текстовый запрос	Выполняет поиск (BM25 + Jina Reranker)
3	Задаёт диапазоны цены, площади, цены за сотку, минимальные скоры, локацию	Применяет фильтры через MongoDB-запрос с операторами \$gte/\$lte/\$in

Шаг	Пользователь	Система
4	Выбирает сортировку (по дате, цене, площади, скорам)	Перестраивает выдачу sort + skip + limit
5	Нажимает на карточку	Переходит к UC2

Альтернативные сценарии: пустой запрос → каталог в обычном режиме; нет подходящих участков → «Нет объявлений»; сервер недоступен → сообщение об ошибке.

UC2 — просмотр деталей участка (пользователь)

Предусловия: участок существует в базе.

Шаг	Пользователь	Система
1	Открывает карточку участка	Делает findOne по _id + \$geoNear к infra_objects для расчёта расстояний
2	Просматривает данные	Отображает название, локацию, цену, площадь, описание, теги характеристик, 4 индикатора скоров, график истории цены, таблицу расстояний до 8 типов инфраструктуры, координаты, профиль продавца

Альтернативный сценарий: участок удалён → «Не найдено»; сервер недоступен → ошибка.

UC3 — просмотр карты участков (пользователь)

Шаг	Пользователь	Система
1	Нажимает «Карта»	Прогрессивно загружает участки порциями по 200,

Шаг	Пользователь	Система
		отображает их в Leaflet с кластеризацией
2	Видит маркеры разных цветов	Маркеры окрашиваются по total_score: зелёный / жёлтый / красный
3	Может вводить текстовый запрос, фильтр по score	Перестраивает выдачу
4	Нажимает на маркер	Показывает попап с краткой информацией и ссылкой «Подробнее →» (UC2)

UC4 — переход на Авито (пользователь)

Действие: клик по ссылке «Посмотреть на Авито» → открытие исходного объявления в новой вкладке. Запросов к БД не выполняется.

III. Анализ данных

UC10 — просмотр статистики (администратор)

Предусловия: администратор авторизован; открыта «Панель данных».

Шаг	Пользователь	Система
1	Открывает панель	Автоматически запрашивает количество документов в каждой коллекции
2	Видит таблицу статистики	Показывает количество объявлений, метро, больниц, школ, детсадов, магазинов, ПВЗ, остановок, негативных объектов
3	Нажимает «Обновить»	Перезапрашивает countDocuments

Дополнительно поддерживается расчёт средних значений (средняя цена / сотка), распределений баллов и графиков, которые отображаются на странице «Статистика».

IV. Экспорт данных

UC8 — экспорт данных (администратор)

Шаг	Пользователь	Система
1	Нажимает «Экспортировать всё»	Формирует JSON со всеми коллекциями (plots, infra_objects, users)
2	Подтверждает скачивание	Браузер сохраняет файл land_plots_export_YYYY-MM-DD.json

Альтернативные сценарии: база пуста → пустые массивы в JSON; сервер недоступен → ошибка.

Вывод о преобладании операций

Для разработанного решения преобладают операции чтения:

- основной интерфейс — каталог, карта и карточки участков — это исключительно find-запросы, выполняемые многократно для каждого пользовательского сеанса;
- запись данных производится либо администратором (импорт, управление инфраструктурой), либо владельцем участка (CRUD своего объявления), что происходит на порядки реже, чем чтения;
- источник данных — парсер Авито — пишет в БД пакетно (upsert по avito_id), нагрузка размазана во времени и не блокирует основные сценарии.

Поэтому модель оптимизирована под чтение: хранятся предрасчитанные feature_score, infra_score, negative_score, total_score и features_text — это даёт

$O(\log N)$ на каталог с сортировкой и $O(1)$ на детальную карточку при минимуме JOIN-подобных операций.

4. Модель данных

4.1. Нереляционная модель данных (MongoDB)

Графическое представление

Схема нереляционной модели (документная) приведена на рисунке 2. Три коллекции: `plots`, `infra_objects`, `users`. Связь между `plots` и `infra_objects` — гео-связь через `$geoNear`; связь `plots.owner_id` → `users._id` — мягкая ссылка по строковому идентификатору.

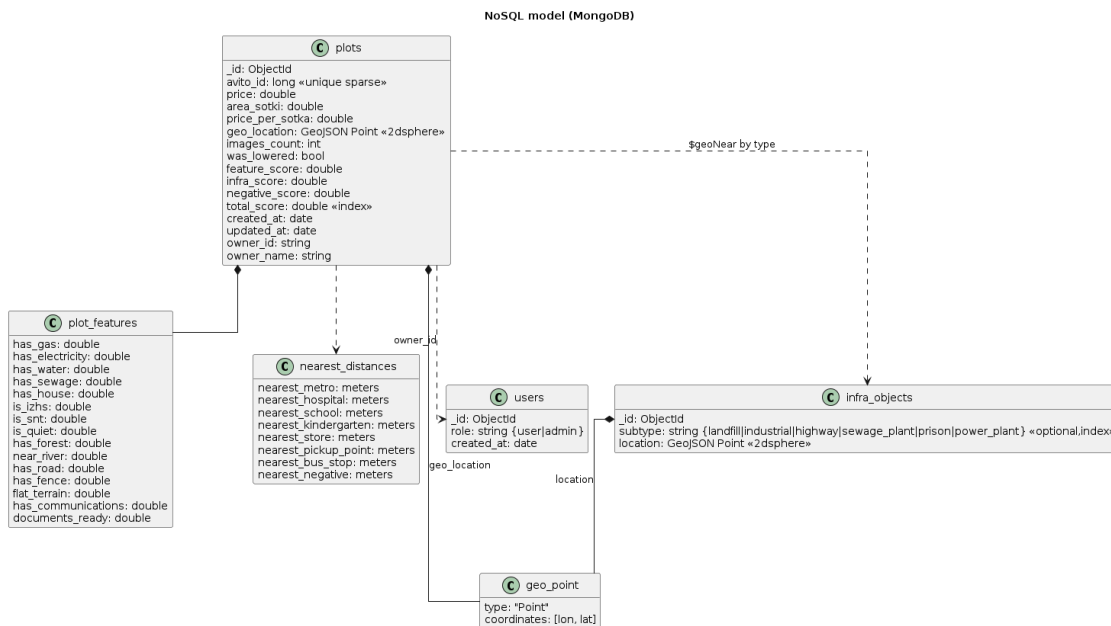


Рисунок 2. Схема нереляционной модели данных (MongoDB).

Описание назначений коллекций, типов данных и сущностей

В модели определены три коллекции.

Коллекция `plots` — объявления о продаже земельных участков. Каждый документ — одно объявление вместе с уже рассчитанной аналитикой:

- `_id` (ObjectId, 12 байт) — идентификатор в MongoDB;
- `avito_id` (Int64, 8 байт) — идентификатор объявления на Авито (unique sparse);
- `title` (String, до 100 символов), `description` (String, до 80 000 символов) — заголовок и полный текст;

- price, area_sotki, price_per_sotka (Double, по 8 байт) — числовые характеристики;
- location, address, geo_ref — текстовое описание расположения (район, адрес, гео-метка);
- geo_location (GeoJSON Point, 58 байт) — {type:"Point", coordinates:[lon, lat]} для 2dsphere-индекса;
- url, thumbnail, images_count, was_lowered — медиа и метаданные;
- features — вложенный объект из 15 полей типа Double (has_gas, has_electricity, has_water, has_sewage, has_house, is_izhs, is_snt, is_quiet, has_forest, near_river, has_road, has_fence, flat_terrain, has_communications, documents_ready) — вероятностные оценки (0.0–1.0) для каждой текстовой характеристики;
- feature_score, features_text — агрегаты по фичам;
- infra_score, negative_score, total_score (Double) — итоговые баллы (0–1);
- created_at, updated_at (Date) — отметки времени;
- owner_id, owner_name — денормализация ссылки на пользователя.

Индексы: geo_location (2dsphere), avito_id (unique sparse), price, area_sotki, total_score.

Коллекция infra_objects — инфраструктурные и негативные объекты. Единая коллекция содержит и обычную инфраструктуру, и негативные объекты — расширение модели сводится к добавлению нового значения type:

- _id (ObjectId), name (String, ~37 байт), type (enum: metro_station, hospital, school, kindergarten, store, pickup_point, bus_stop, negative);
- subtype (enum, только для type="negative": landfill, industrial, highway, sewage_plant, prison, power_plant);
- location (GeoJSON Point).

Индексы: location (2dsphere), составной B-tree type + name, отдельный subtype.

Коллекция users — пользователи системы:

- `_id` (ObjectId, 12 байт), `username` (String, до 50), `password_hash` (PBKDF2-SHA256, 64 байта), `role` ("user"/"admin"), `created_at` (Date).

Индекс: `username` (unique).

Оценка удельного объёма информации

Размер одного документа `plots`. Подсчёт ведём по фактическим данным (2174 записи в БД):

Группа полей	Среднее, байт
Служебное: <code>_id</code> , <code>avito_id</code>	20
Текст: <code>title</code> , <code>description</code> , <code>location</code> , <code>address</code> , <code>geo_ref</code>	1228
Числовые: <code>price</code> , <code>area_sotki</code> , <code>price_per_sotka</code>	24
Гео: <code>geo_location</code>	58
Медиа: <code>url</code> , <code>thumbnail</code> , <code>images_count</code> , <code>was_lowered</code>	277
Фичи: <code>features</code> (15×Double) + <code>feature_score</code> + <code>features_text</code>	322
Скоры: <code>infra_score</code> , <code>negative_score</code> , <code>total_score</code>	24
Мета: <code>created_at</code> , <code>updated_at</code> , <code>owner_id</code> , <code>owner_name</code>	50
BSON-оверхед (ключи, типы, ~350 длины)	
Итого средний документ	~2353

Обозначим N — число объявлений, I — число инфраструктурных объектов, U — число пользователей.

Объём коллекций (без индексов):

$$S_{plots}(N) = 2353 \cdot N \text{ байт}$$

$$S_{infra}(I) = 145 \cdot I \text{ байт}$$

$$S_{users}(U) = 99 \cdot U \text{ байт}$$

Размеры индексов: plots — ~400 байт/запись (включая 2dsphere ~200 байт), infra_objects — ~300 байт/объект, users — ~40 байт/пользователя. С учётом индексов:

$$S_{total}(N, I, U) = 2753 \cdot N + 445 \cdot I + 139 \cdot U \text{ байт}$$

Пример. При $N = 2174$, $I = 87$, $U = 10$:

- plots + индексы: $2174 \times 2753 \approx 5.99$ МБ
- infra_objects + индексы: $87 \times 445 \approx 37.8$ КБ
- users + индексы: $10 \times 139 \approx 1.4$ КБ
- Общий объём: ≈ 6.0 МБ

Главный вектор роста — N (количество объявлений). Каждый новый участок добавляет ~2.75 КБ. При $N = 100000$ объём составит около 263 МБ. Инфраструктура и пользователи растут заметно медленнее.

Избыточность модели

«Чистый» документ (поля, непосредственно описывающие участок без производных и BSON-оверхеда): ~1595 байт. Коэффициент избыточности:

$$R(N) = \frac{2753N + 40105}{1595 \cdot N} = 1,73 + \frac{25,1}{N}$$

При больших N избыточность стремится к $\approx 1.73 \times$. Основной вклад дают: BSON-оверхед (~370 байт), денормализованные текстовые и агрегированные фичи (features_text, feature_score) и предрасчитанные скорости — это плата за $O(\log N)$ при поиске и сортировке.

Запросы к модели

Для каждого ключевого юзкейса приведём mongo-запрос.

Q1. Каталог с фильтрами и сортировкой (UC1):

```
db.plots.countDocuments({
  price: { $gte: 500000, $lte: 3000000 },
  area_sotki: { $gte: 5 }
});
```

```
db.plots.find(
  { price: { $gte: 500000, $lte: 3000000 }, area_sotki: { $gte: 5 } }
).sort({ total_score: -1 }).skip(0).limit(20);
```

Q2. Поиск с ранжированием (UC1):

```
db.plots.find(
  { price: { $gte: 500000, $lte: 3000000 }, area_sotki: { $gte: 5 } },
  { title: 1, description: 1, price: 1, total_score: 1, features_text: 1 }
).sort({ total_score: -1 }).limit(100);
```

Q3. Детали участка (UC2):

```
db.plots.findOne({ _id: ObjectId("...") });
```

```
db.infra_objects.aggregate([
  { $geoNear: { near: { type: "Point", coordinates: [lon, lat] },
    distanceField: "dist_meters", spherical: true } },
  { $sort: { type: 1, dist_meters: 1 } },
  { $group: { _id: "$type", name: { $first: "$name" }, dist_meters: { $first:
"$dist_meters" } } }
]);
```

Q4. Маркеры карты (UC3):

```
db.plots.find({}, { title:1, price:1, geo_location:1, total_score:1, features_text:1 })
.skip(0).limit(200);
```

Q5. Создание объявления (UC5): \$geoNear к infra_objects → insertOne в plots (2 запроса).

Q9. Экспорт данных (UC8): db.plots.find({}), db.infra_objects.find({}), db.users.find({}) — 3 запроса.

Q10. Импорт (UC9): `db.plots.updateOne({avito_id: ...}, {$set: {...}}, {upsert: true})` для каждого объявления.

Q11. Статистика (UC10): `countDocuments` по каждой коллекции — 3 запроса.

4.2. Реляционная модель данных (PostgreSQL + PostGIS)

Графическое представление

Схема реляционной модели приведена на рисунке 3.

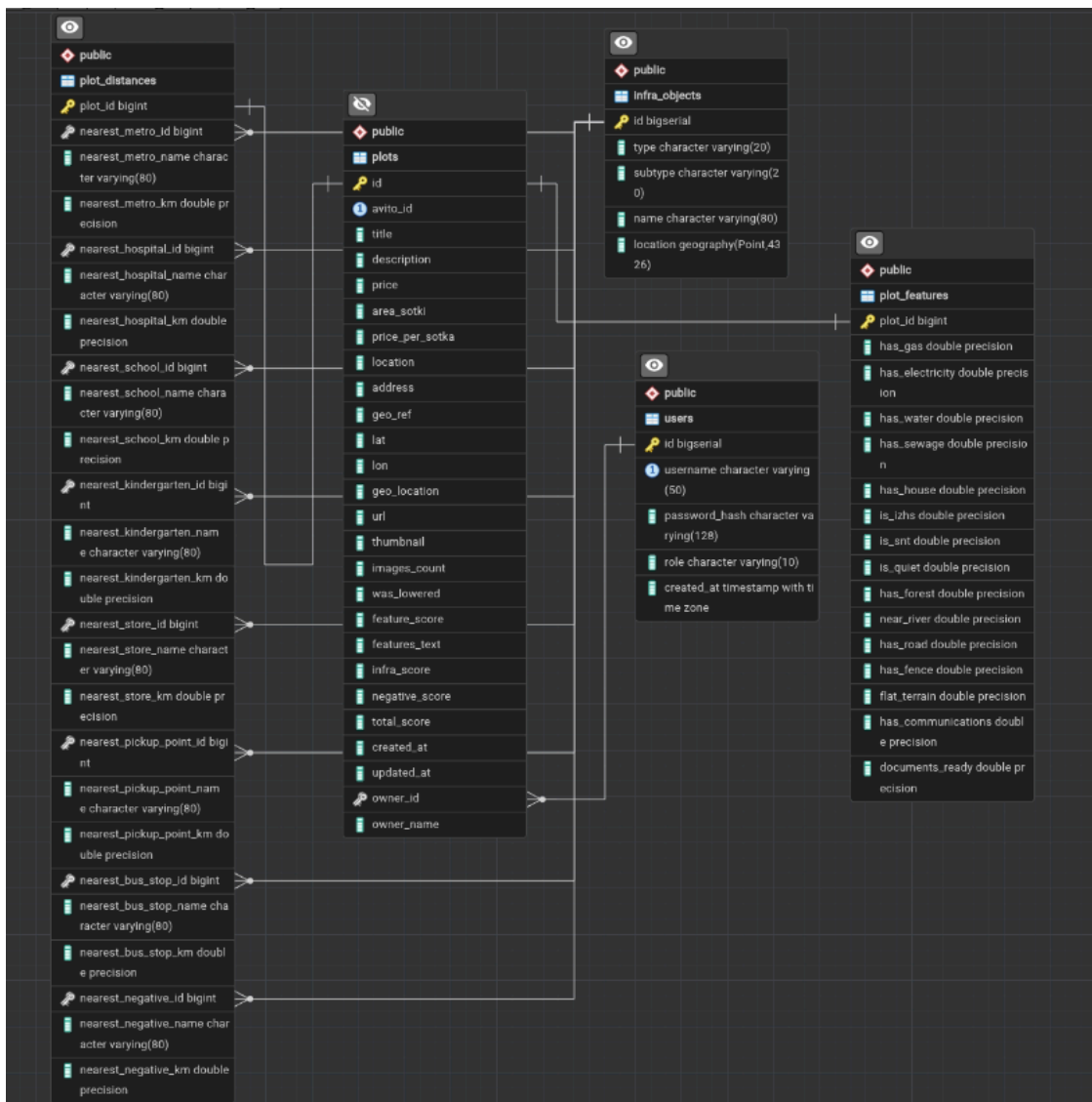


Рисунок 3. Схема реляционной модели данных (PostgreSQL + PostGIS).

Описание таблиц

Для реляционной модели используется 5 таблиц: `plots`, `plot_features` (15 фич, 1:1 с `plots`), `plot_distances` (8 пар «имя+км», 1:1 с `plots`), `users` и единая `infra_objects` с

type/subtype. Геополе хранится через PostGIS (geography/geometry с GiST-индексом).

Средние размеры строк (с учётом BSON-аналога — заголовок строки PostgreSQL ~80 байт): plots ~1963, plot_features ~160, plot_distances ~520, users ~190, infra_objects ~160.

Оценка объёма

Без индексов:

$$S_{data,rel}(N) = (1963 + 160 + 520) \cdot N + 160 \cdot I + 190 \cdot U = 2643N + 160I + 190U$$

С учётом B-tree + GiST индексов (плотно ~600 байт/участок, 240 байт/инфра-объект, 80 байт/пользователя):

$$S_{total,rel}(N, I, U) = 3243 \cdot N + 400 \cdot I + 270 \cdot U$$

«Чистый» объём — 1553 байт/участок. Избыточность:

$$R_{rel}(N) = \frac{3243N + 37500}{1553 \cdot N} = 2,09 + \frac{24,1}{N}$$

Примеры запросов

Q1. Каталог с фильтрами:

```
SELECT COUNT(*) FROM plots
WHERE price BETWEEN 500000 AND 3000000 AND area_sotki >= 5;
```

```
SELECT id, title, price, area_sotki, total_score
FROM plots
WHERE price BETWEEN 500000 AND 3000000 AND area_sotki >= 5
ORDER BY total_score DESC
LIMIT 20 OFFSET 0;
```

Q3. Детали участка:

```
SELECT p.*, pf.*, pd.*
FROM plots p
LEFT JOIN plot_features pf ON pf.plot_id = p.id
```

LEFT JOIN plot_distances pd ON pd.plot_id = p.id

WHERE p.id = 12345;

Q5. Создание участка (геообогащение):

WITH p AS (

SELECT ST_SetSRID(ST_MakePoint(29.88, 59.84), 4326)::geography AS p_in
) , ranked AS (

SELECT i.type, i.name,

ST_Distance(p.p_in, i.location) / 1000.0 AS km,

ROW_NUMBER() OVER (PARTITION BY i.type ORDER BY i.location <->
p.p_in) AS rn

FROM infra_objects i CROSS JOIN p

)

SELECT type, name, km FROM ranked WHERE rn = 1;

INSERT INTO plots (...) VALUES (...) RETURNING id;

INSERT INTO plot_features (plot_id, ...) VALUES (...);

INSERT INTO plot_distances (plot_id, ...) VALUES (...);

4.3. Сравнение моделей

Удельный объём информации

Сравним суммарные формулы при $I = 87$, $U = 10$:

- нереляционная: $S_{doc}(N) = 2753N + 40105$
- реляционная: $S_{rel}(N) = 3243N + 37500$

Разность:

$$\Delta S(N) = S_{rel}(N) - S_{doc}(N) = 490N - 2615$$

При практических масштабах:

N	S_{doc} , байт	S_{rel} , байт	Экономия NoSQL, байт
1 000	2 793 105	3 280 500	487 395
2 174	6 027 327	7 087 782	1 060 455
10 000	27 570 105	32 467 500	4 897 395

Нереляционная модель компактнее за счёт того, что 15 фич и 8 расстояний хранятся вложенными, тогда как реляционная требует двух дополнительных таблиц с РК/ФК и индексами по plot_id.

Запросы по отдельным юзкейсам

Сложность ключевых запросов в обеих моделях приведена в таблице 1. Через N обозначено число объявлений, I — число инфра-объектов, U — пользователей, M — размер пакета при импорте, K — число типов инфраструктуры (в текущей реализации $K = 8$).

Таблица 1. Сложность ключевых операций по юзкейсам.

Use Case	Описание	NoSQL:			Асимптотика
		SQL: запросы / таблицы	запросы коллекции	/	
UC1	Поиск и фильтрация	2 / 1	2 / 1		$O(\log N)$ за счёт B-tree-индексов по price, area_sotki, total_score
UC2	Детали участка	1 / 3 (JOIN)	2 / 2 (с \$geoNear)	(с \$geoNear по 2dsphere)	$O(1) + O(K \log I)$ за \$geoNear по 2dsphere

Use Case	Описание	NoSQL:			Асимптотика
		SQL: запросы / таблицы	запросы	коллекции /	
UC3	Карта (батч 200)	$1 + \lceil N/200 \rceil$ / 1	$1 + \lceil N/200 \rceil$ / 1		$O(N)$ суммарно
UC5	Создание участка	4 / 4	2 / 2		$O(K \log I)$ за обогащение
UC6	Редактирование	3–5 / до 3	3–4 / 1		$O(K \log I)$ при изменении координат
UC7	Удаление	2 / до 3	2 / 1		$O(\log N)$
UC8	Экспорт	5 / 5	3 / 3		$O(N + I + U)$
UC9	Импорт пакета M	$4 \cdot M / 4$	$2 \cdot M / 2$		$O(M \cdot K \log I)$
UC10	Статистика	5 / 5	3 / 3		$O(1)$ через countDocuments
UC11	Удаление любого	2 / 3	2 / 1		$O(\log N)$

Оценка производительности на типовой выборке

В таблице 2 приведены измеренные значения времени выполнения ключевых запросов на текущей выборке (2174 участка, 87 инфра-объектов, локальный MongoDB 7 в контейнере). Замеры — средние по 50 повторам, прогретые индексы.

Таблица 2. Время выполнения запросов NoSQL (MongoDB) на текущей выборке.

Запрос	Использует индекс	Среднее время, мс
Q1. Каталог фильтрами сортировка top-20	с price, area_sotki, + total_score	~12
Q3. findOne участка по _id	_id	~2
Q3.b. \$geoNear инфраструктуры	location (2dsphere)	~28
Q4. Маркеры карты (batch 200)	_id	~9
Q5. Создание участка (обогащение + insert)	location (2dsphere)	~45
Q11. countDocuments по 3 коллекциям	мета-счётчик	~3

Видно, что наиболее «дорогими» остаются операции с участием \$geoNear — они линейны по количеству типов инфраструктуры K и логарифмичны по I за счёт 2dsphere-индекса. При этом результат прирастает в plots уже денормализованно, поэтому повторные чтения карточки не требуют повторного гео-обогащения.

Вывод — что лучше, SQL или NoSQL

По итогам сравнения обе модели рабочие, но для данного проекта NoSQL (MongoDB) подходит лучше по следующим причинам.

1. Профиль нагрузки. Преобладают чтения; запросы к участку нужны вместе с фичами и расстояниями — в NoSQL это один документ, в SQL — три таблицы и JOIN.
2. Геопоиск. MongoDB предоставляет \$geoNear и 2dsphere-индекс «из коробки», в PostgreSQL для аналогичного функционала требуется расширение PostGIS.

3. Расход места. Разница в коэффициенте при N составляет 490 байт/документ в пользу нереляционной модели; при 100 000 участков экономия ~ 49 МБ + $4\times$ меньше места под индексы вложенных таблиц.
4. Гибкость схемы. Список фич и типов инфраструктуры вероятно будет расширяться. В MongoDB новое поле/type добавляется без миграций; в SQL — ALTER TABLE или новая таблица.

Минусы NoSQL, которые осознанно приняты: - ссылка `plots.owner_id` $\rightarrow$ `users` не контролируется на уровне БД (отсутствует FK); - транзакции возможны только в рамках реплика-сета; используем атомарные операции `updateOne/upsert`.

5. Разработанное приложение

Краткое описание и архитектура

Было реализовано полнофункциональное веб-приложение Land Plots — сервис объявлений о продаже земельных участков с автоматической аналитикой привлекательности. Система позволяет:

- вести каталог участков с фильтрами, сортировкой и пагинацией;
- просматривать карточку участка с интегральным скором, графиком истории цен, таблицей расстояний до 8 типов инфраструктуры и блоком продавца;
- видеть участки на интерактивной карте Leaflet с кластеризацией и цветовой индикацией;
- выполнять гибридный поиск (BM25 + Jina Reranker) по произвольному текстовому запросу;
- создавать, редактировать и удалять собственные объявления (с автоматическим пересчётом фич, расстояний и скоров);
- администрировать инфраструктурные коллекции, импортировать / экспортировать данные и просматривать статистику.

Архитектура

Архитектура слоистая, на бэкенде — гексагональная (Clean Architecture):

- domain/ — сущности (entities.py), use cases (auth.py, data_management.py, infra.py, plots.py, stats.py), интерфейсы репозитория и логика скоринга (scoring.py);
- infrastructure/ — реализация репозитория на Motor (async MongoDB), модуль аутентификации, embeddings-клиент, перанкер;
- interfaces/ — FastAPI-роуты (routes/), pydantic-схемы, зависимости.

Развёртывание через docker-compose.

Слой	Технология
Frontend	React 19, TypeScript, Vite, TailwindCSS, React Query, React Router, Leaflet (react-leaflet с кластеризацией), Recharts (графики), Zustand (state)
Backend	Python 3.11, FastAPI, Motor (async MongoDB driver), Pydantic v2, PyJWT, PBKDF2-SHA256
Поиск / ML	Локальный BM25 (русская токенизация), Jina Embeddings v3 (15 фич участка), Jina Reranker
Гео	MongoDB 2dsphere index, \$geoNear, geopy для вспомогательных вычислений
СУБД	MongoDB 7
Контейнеризация	Docker, docker-compose, nginx (раздача frontend)
Админка	Mongo Express (для отладки)
Тесты / CI	GitHub Actions, pytest

Схема экранов приложения

Переходы между экранами разработанного приложения отображены на рисунке 4.

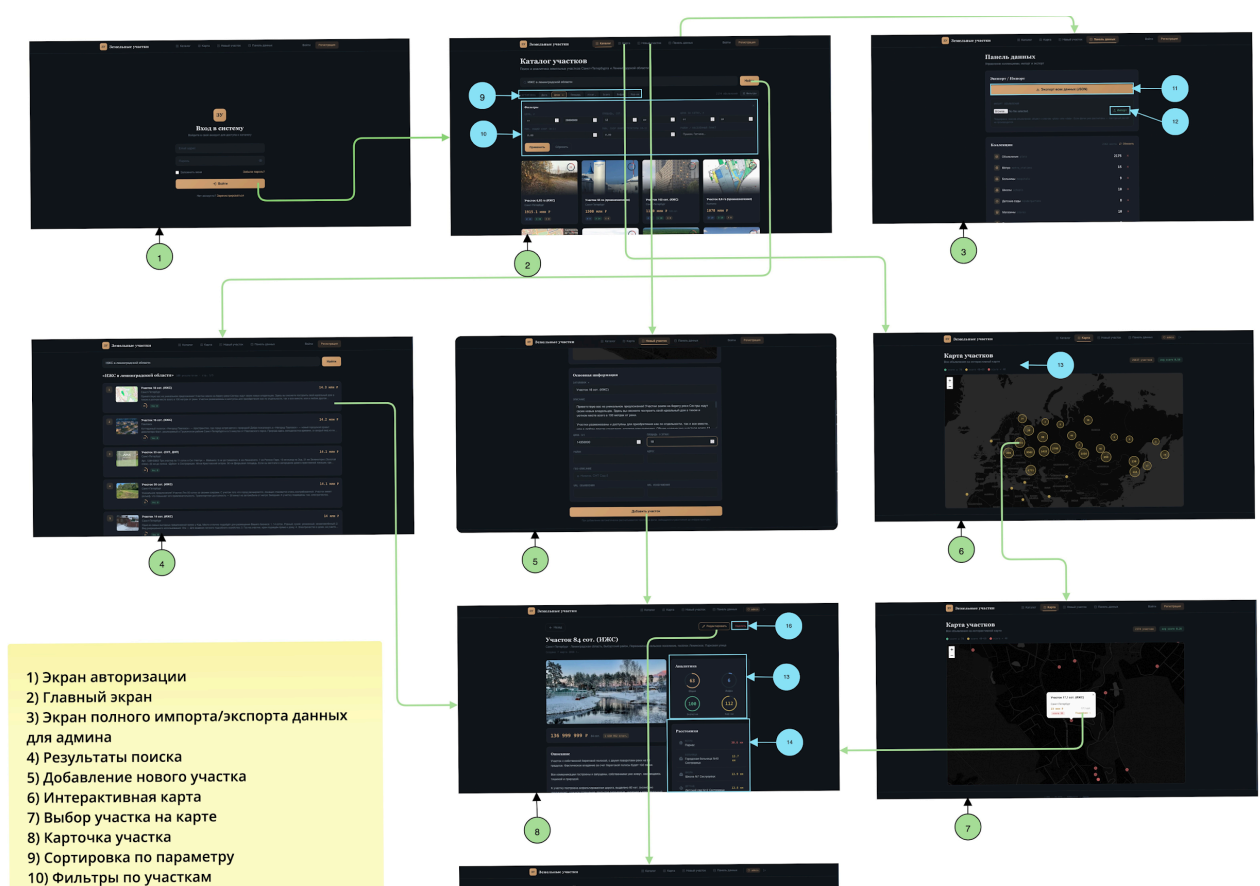


Рисунок 4. Схема экранов приложения и переходов между ними.

Снимки экрана приложения

В репозитории представлены ключевые экраны (см. wiki и каталог docs/):

- лендинг и каталог с фильтрами;
- карта участков с кластеризацией и цветовыми маркерами;
- страница деталей участка с индикаторами скоров и таблицей расстояний;
- форма создания / редактирования участка с интерактивным выбором точки;
- админ-панель: импорт / экспорт, статистика, управление инфраструктурой.

6. Выводы

Достигнутые результаты

В результате работы реализовано веб-приложение, выполняющее все заявленные функции:

- хранение и предрасчитанная аналитика для ~2200 реальных объявлений Авито по Санкт-Петербургу и Ленинградской области;
- гибридный поиск (фильтры + BM25 + семантический реранкер) по каталогу;
- интерактивная карта с прогрессивной загрузкой и кластеризацией;
- CRUD объявлений с автоматическим пересчётом 15 текстовых фич, 8 расстояний и итогового счёта;
- разграничение ролей (user / admin), JWT-аутентификация;
- админка с массовым импортом / экспортом, управлением инфраструктурой и статистикой;
- развёртывание одной командой `docker-compose up --build`.

Спроектирована и обоснована модель данных в двух вариантах (NoSQL + SQL), проведена количественная оценка объёма и избыточности. Показано, что для данного профиля нагрузки документная модель MongoDB даёт более компактное хранение (~490 байт/документ экономии) и проще запросы для ключевых юзкейсов.

Недостатки и пути для улучшения

- Отсутствие FK на уровне БД. Ссылка `plots.owner_id` → `users` контролируется только на уровне приложения. Решение: ввести strict-схему MongoDB с JSON-валидатором и периодическую джобу-валидатор целостности.
- Денормализация продавца. `owner_name` хранится в `plots` для быстрого отображения, но не обновляется при смене имени пользователя. Решение: фоновая задача пересчёта или \$lookup на этапе детальной карточки.

- Холодный старт реранкера. При первом запросе Jina API может отвечать с задержкой. Решение: прогрев на старте + кеш топ-100 на 5 минут.
- Покрывание тестами. Сейчас есть unit-тесты домена и интеграционные тесты роутов, но не покрыты сценарии импорта больших файлов и фоновый пересчёт скоров.
- Текстовый поиск ограничен русским языком. Токенизация настроена под русский; английские объявления ранжируются хуже.

Будущее развитие решения

- Подключение второго источника объявлений (например, Циан) с дедупликацией по координатам.
- Расширение скоринга: учёт уклона рельефа, кадастровых границ, удалённости от пробок (через сторонние API).
- Личный кабинет с «алертами» по новым подходящим участкам (push / Telegram).
- Карта с тепловой картой негативных факторов и слоёв инфраструктуры.
- Перенос реранкера в self-hosted-вариант для снижения зависимости от внешнего API.
- Шардирование plots по геозоне при росте до ~1М объявлений.

7. Приложения

Документация по сборке и развёртыванию приложения

Требования: Docker 24+, docker-compose v2, ~2 ГБ свободного места.

Шаги:

1. Клонировать репозиторий:

```
git clone https://github.com/moevm/nsql1h26-land.git
```

```
cd nsql1h26-land
```

2. (Опционально) подготовить переменные окружения:

```
cd app && cp .env.example .env
```

```
# отредактировать JINA_API_KEY, JWT_SECRET при необходимости
```

```
cd ..
```

3. Поднять стек:

```
docker-compose up --build
```

4. После старта будут доступны:

- Frontend: <http://localhost:3000>
- Backend API + Swagger UI: [http://localhost:8000 \(/docs\)](http://localhost:8000 (/docs))
- Mongo Express: <http://localhost:8081>
- MongoDB: localhost:27017

При первом запуске автоматически создаются тестовые учётные записи: admin/admin и user/user. Дамп данных (2174 объявления + 87 инфра-объектов) восстанавливается из app/dump/ скриптом app/mongo-init/restore.sh.

Инструкция для пользователя

1. Открыть <http://localhost:3000>.

2. Без логина: доступны каталог, карта, поиск, страница деталей и переход на Авито.
3. Войти как user/user для добавления своих объявлений, редактирования и удаления своих карточек.
4. Войти как admin/admin для доступа к «Панели данных»: статистика, импорт / экспорт, управление инфраструктурой, удаление любого участка.
5. Для импорта подготовить JSON одного из форматов:
 - массив объектов: [{...}, {...}]
 - объект с ключом: {"plots": [...]} или {"data": [...]}
6. После импорта скоры пересчитываются автоматически в фоне; прогресс виден в статистике.

8. Используемая литература

1. Репозиторий проекта: <https://github.com/moevm/nsql1h26-land>

2. Документация MongoDB: <https://www.mongodb.com/docs/manual/>
3. MongoDB Geospatial Queries / 2dsphere: <https://www.mongodb.com/docs/manual/geospatial-queries/>
4. FastAPI: <https://fastapi.tiangolo.com/>
5. Motor (async MongoDB driver): <https://motor.readthedocs.io/>
6. Jina Embeddings v3 API: <https://jina.ai/embeddings/>
7. Jina Reranker API: <https://jina.ai/reranker/>
8. React 19 docs: <https://react.dev/>
9. Leaflet: <https://leafletjs.com/>
10. PostGIS reference (для сравнения с SQL-моделью): <https://postgis.net/docs/>
11. Методика расчёта объёма модели данных (МО ЭВМ ЛЭТИ): http://se.moevm.info/doku.php/staff:courses:no_sql_introduction:calculating_data_model_size